ЗВІТ

про виконання лабораторної роботи №6

«RESTful API»

з дисципліни

«Вебтехнології»

Студента групи КН-2327Б

Багрія-Белза Андрія

Мета роботи: Реалізувати RESTful API /api: JSON-інтерфейс для читання та управління даними системи з автентифікацією за сесією, матрицею ролей 1:1 з web і тими самими правилами валідації (без подвійної бізнес-логіки).

Завдання 1: Автентифікація та контроль ролей для /api: та сама сесія (cookie), що і для Web; гостю — 401, неповноваженій ролі — 403 зі списком потрібних ролей; неіснуюча точка — JSON 404.

1. src/api/http.ts — middleware захисту /api

```
/** Захист /api ролями: 401 для гостей, 403 для низьших ролей (JSON, без редіректів). */
export function apiAuth(roles: readonly string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    if (!req.user) return res.status(401).json({ error: "Authentication required" });
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: "Forbidden", requiredRoles: roles });
    }
    next();
  };
}
```

2. Жива поведінка (guest / visitor / admin):

```
GET /api/works?pageSize=2 (guest)
HTTP 200 {"data":[{"id":"…","name":"Відновлення даних","basePrice":"2000","baseHours":3},
                  {"id":"…","name":"Встановлення ОС","basePrice":"700","baseHours":1.5}],
          "page":1,"pageSize":2,"total":14,"totalPages":7}

POST /api/clients (guest)
HTTP 401 {"error":"Authentication required"}

POST /api/clients (visitor)
HTTP 403 {"error":"Forbidden","requiredRoles":["ADMIN","MANAGER"]}

GET /api/clients/no-such-id (admin)
HTTP 404 {"error":"Not found"}
```

Завдання 2: Ресурси та JSON-конверти: GET-списки з пагінацією ?page&pageSize, картки з повними зв'язками, коди 201/200/204; помилки валідації — 400 зі списком errors[]. REST не дублює бізнес-логіку: мутації викликають ті самі web-контролери (runWeb).

1. src/api/routes.ts — фрагмент

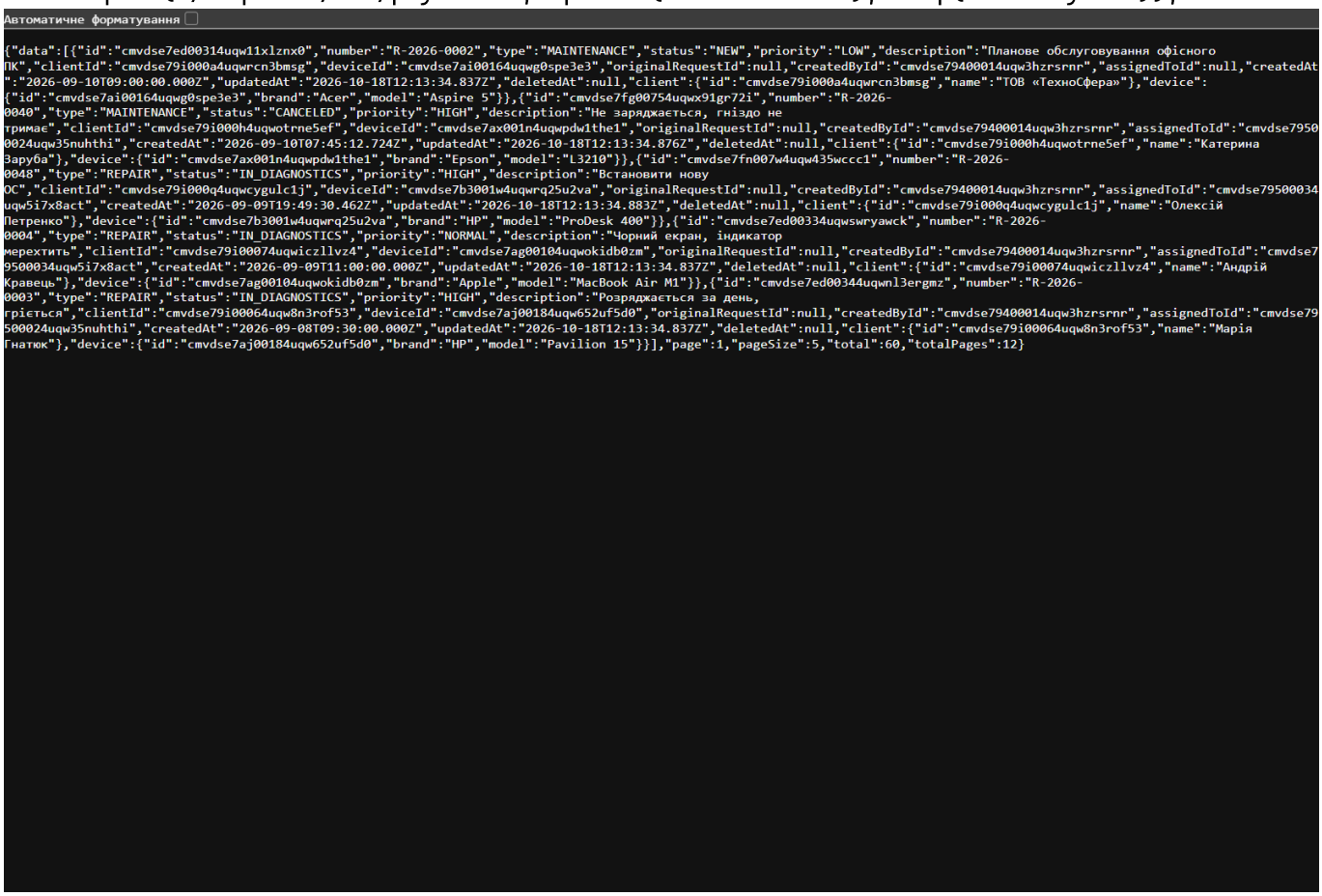
```
// Публічні читання (усі ролі, включно з гостями)
router.get("/requests", wrap(L.listRequests));
router.get("/requests/:id", wrap(L.getRequest));

// Клієнти / Пристрої (MANAGER+)
router.post("/clients", apiAuth(ROLES.MANAGER), wrap(M.createClient));
router.put("/clients/:id", apiAuth(ROLES.MANAGER), wrap(M.editClient));
router.delete("/clients/:id", apiAuth(ROLES.MANAGER), wrap(M.removeClient));

// Заявки — Статус / роботи / запчастини / діагностика — МАЙСТЕР+
router.post("/requests/:id/transition", apiAuth(ROLES.TECH), wrap(M.transitionRequest));
router.post("/requests/:id/works", apiAuth(ROLES.TECH), wrap(M.addWorkLine));
router.post("/requests/:id/parts", apiAuth(ROLES.TECH), wrap(M.addPartLine));
router.post("/requests/:id/diagnostics", apiAuth(ROLES.TECH), wrap(M.saveDiagnostics));

// Оцінка / рахунок / оплата / гарантія — МЕНЕДЖЕР+
router.post("/requests/:id/estimate", apiAuth(ROLES.MANAGER), wrap(M.saveEstimate));
```

```
router.post("/requests/:id/invoice", apiAuth(ROLES.MANAGER), wrap(M.createInvoice));
router.post("/requests/:id/payments", apiAuth(ROLES.MANAGER), wrap(M.addPayment));
```



Список /api/requests?pageSize=5 у браузері: конверт {data, page, pageSize, total, totalPages} — 5 із 60 заявок. Картка /api/requests/:id повертає {data: …} з клієнтом, пристроєм, автором, роботою, запчастини, оцінкою, рахунком, коментарями та вкладеннями. Парольні хеші у зв'язках користувачів не повертаються (Prisma omit: password).

Завдання 3: Повний життєвий цикл заявки через API: створення → перехід до діагностики → роботи/запчастини → діагностика → оцінка (DRAFT → APPROVED) → рахунок → оплати (UNPAID → PARTIAL → PAID).

1. Живий прогін (admin / manager / tech1):

```
POST /api/clients (admin)
HTTP 201 {"data":{"id":"…","name":"Report Client 153045","phone":"0675550001", …}}

POST /api/devices (admin)
HTTP 201 {"data":{"id":"…","type":"PC","brand":"Report","model":"R153045", …}}

POST /api/requests (admin)
HTTP 201 {"data":{"id":"…","number":"R-2026-0061","type":"REPAIR","status":"NEW", …}}

POST /api/requests/:id/transition (tech1) {"status":"IN_DIAGNOSTICS"}
HTTP 200 {"data":{…,"status":"IN_DIAGNOSTICS"}}

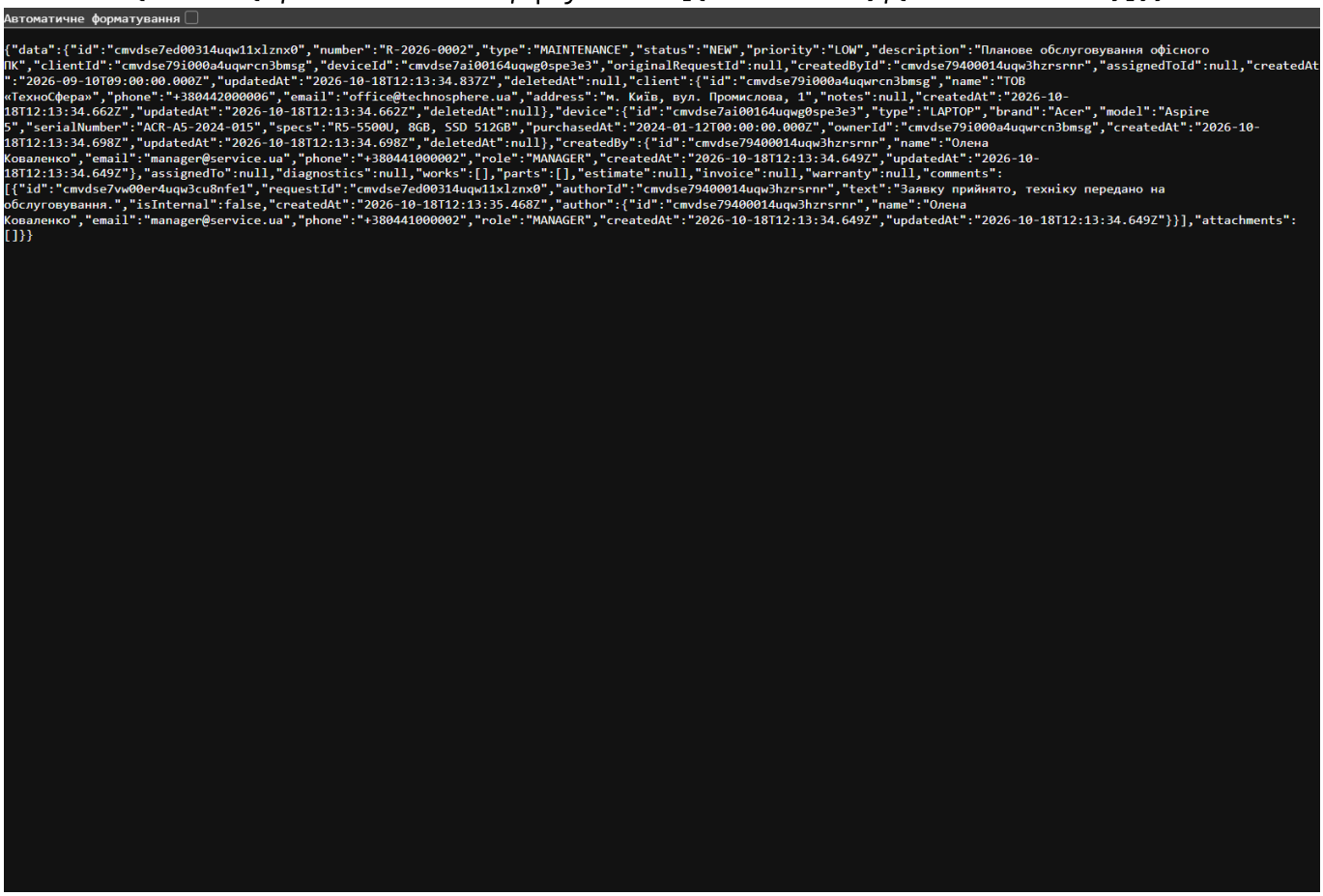
POST /api/requests/:id/works (tech1) → HTTP 201 (робота «Відновлення даних», 2000 грн, 3 год)
POST /api/requests/:id/parts (tech1)  → HTTP 201 (RAM DDR4 16GB × 2 × 10.50)
POST /api/requests/:id/diagnostics (tech1) → HTTP 200 (findings + вартість 5)

POST /api/requests/:id/estimate (manager)
{"status":"DRAFT","totalWorks":"2000","totalParts":"21","total":"2021"}
HTTP 200
POST /api/requests/:id/estimate/approve (manager)
HTTP 200 {"data":{…,"status":"APPROVED","approvedAt":"…"}}
```

```
POST /api/requests/:id/invoice (manager) {"discount":0}
HTTP 201 {"data":{…,"finalTotal":"2021","status":"UNPAID","payments":[]}}

POST /api/requests/:id/payments (manager) {"amount":5.25,"method":"CASH"}
HTTP 201 {"data":{…,"status":"PARTIAL","payments":[{…,"amount":"5.25","method":"CASH"}]}}

POST /api/requests/:id/payments (manager) {"amount":2015.75,"method":"CARD"}
HTTP 201 {"data":{…,"status":"PAID","payments":[{…5.25 CASH…},{…2015.75 CARD…}]}}
```



Повна картка заявки після завершення оплати: статус IN_DIAGNOSTICS, рахунок на 2021 грн зі статусом PAID та дві оплати (5.25 CASH + 2015.75 CARD). Цикл пройшов через ті самі web-контролери (runWeb), тож статусна машина та інваріанти I1–I10 застосовувалися так само, як і через Web.

Завдання 4: Автоматична перевірка API: E2E-скрипт на живій базі (статуси, створення, цикл, пагінація, видалення).

Скрипт scripts/lab6-smoke.ps1 автоматизовано перевіряє 70 сценаріїв: 401 для гостей, 403 за ролями (requiredRoles), 404, 400 зі списком помилок, створення клієнта/пристрою/заявки, повний фінансовий цикл (UNPAID → PARTIAL → PAID), пагінацію списків і видалення записів (204). Результат: 70 із 70 перевірок пройдено на свіжій базі даних.

Висновок: На цій лабораторній роботі я реалізував RESTful API /api над існуючим застосунком: 36 точок для восьми типів ресурсів із матрицею ролей, ідентичною web-шару, JSON-конвертами {data, page, pageSize, total, totalPages}, пагінацією ?page&pageSize та кодами 401/403/404/400/201/200/204. Ключове рішення — REST не дублює бізнес-логіку: мутації викликають ті самі web-контролери (runWeb), тому валідація, статусна машина та інваріанти однаково діють і для Web, і для API. Реалізовано перехоплення помилок Prisma (P2002/P2003 → 400, P2025 → 404) та

приховування парольних хешів у зв'язках. Результат перевірено живим прогромом повного життєвого циклу заявки (створення → діагностика → оцінка → рахунок → дві оплати → PAID) та E2E-скриптом: 70 із 70 перевірок.